

Introdução: Algoritmos e Programação

Yuri Kaszubowski Lopes

UDESC

Problema

- O que é um problema?

Problema

- O que é um problema?
 - ▶ Relação binária entre **entradas** e **saídas** corretas
 - ▶ Usualmente não especificamos todas as relações (são muitas!)
 - ▶ É dado um predicado verificável (uma propriedade) que as saídas corretas devem satisfazer
 - ▶ Estudamos problemas considerando um espaço de entrada grande e generalista
 - ★ Não generalista: Dentre os alunos desta turma há um par com a mesma data de aniversário?
 - ★ Generalista: Dado um conjunto de n estudantes, existe um par de estudantes com o mesmo aniversário?

Problema

- O que é um problema?
 - ▶ Relação binária entre **entradas** e **saídas** corretas
 - ▶ Usualmente não especificamos todas as relações (são muitas!)
 - ▶ É dado um predicado verificável (uma propriedade) que as saídas corretas devem satisfazer
 - ▶ Estudamos problemas considerando um espaço de entrada grande e generalista
 - ★ Não generalista: Dentre os alunos desta turma há um par com a mesma data de aniversário?
 - ★ Generalista: Dado um conjunto de n estudantes, existe um par de estudantes com o mesmo aniversário?
 - ★ Se considerado apenas dia/mês, com $n > 366$: sempre verdade pelo princípio das casas dos pombos

Problema

- O que é um problema?
 - ▶ Relação binária entre **entradas** e **saídas** corretas
 - ▶ Usualmente não especificamos todas as relações (são muitas!)
 - ▶ É dado um predicado verificável (uma propriedade) que as saídas corretas devem satisfazer
 - ▶ Estudamos problemas considerando um espaço de entrada grande e generalista
 - ★ Não generalista: Dentre os alunos desta turma há um par com a mesma data de aniversário?
 - ★ Generalista: Dado um conjunto de n estudantes, existe um par de estudantes com o mesmo aniversário?
 - ★ Se considerado apenas dia/mês, com $n > 366$: sempre verdade pelo princípio das casas dos pombos
 - ★ Assumir espaço de entrada maior (ano, horário, cidade, ...)

Algoritmo

- procedimento mapeando cada entrada para uma **única** saída (determinístico)
- O algoritmo resolve um problema se retorna uma saída correta para todas as entradas do problema
- Exemplo no problema do aniversário:
 - ▶ Mantenha um registro de nomes (e outro identificador único: e.g., CPF) e o aniversário (inicialmente vazio)
 - ▶ Entreviste cada estudante
 - ★ Se aniversário existe no registro: retorne o par encontrado
 - ★ Senão: adicione os dados no registro
 - ▶ Retorne `None` se o último estudante foi entrevistado sem sucesso

Corretude

- Programas/Algoritmos possuem um tamanho fixo: como provar que são corretos?
- Para pequenas entradas: analisar casos (de teste)
- Para entradas arbitrariamente grandes: algoritmo deve ser **recursivo** ou alguma forma de iteração (loop)
- Devemos induzir (por que a recursão é tão importante em ciência da computação?)

Eficiência

- O quão rápido podemos produzir uma saída correta
 - ▶ Pode ser medido em tempo, mas desejamos independência de máquina
 - ▶ Ideia: Contar o número de operações de tempo fixo que o algoritmo toma para retornar
 - ▶ É esperado que dependa do tamanho da entrada
 - ★ Entrada maior sugere tempo maior
 - ★ Tamanho da entrada geralmente (mas não sempre) chamado de n
 - ▶ Eficiente se retorna em tempo polinomial em função do tamanho da entrada
 - ▶ Algumas vezes não existe algoritmo eficiente para um problema.

Notação Assintótica

- Ignorar fatores constantes e termos de ordem inferiores
- Limites:
 - ▶ Superior: O
 - ▶ Inferior: Ω
 - ▶ Justo: Θ
- Tempo estimado na tabela abaixo considerando uma operação por ciclo em uma máquina de único núcleo com 1 *GHz*
- Número estimado de partículas no universo: $< 10^{100}$

entrada	constante	logarítmico	linear	log-linear	quadrático	polinomial	exponencial
n	$\Theta(1)$	$\Theta(\lg n)$	$\Theta(n)$	$\Theta(n \lg n)$	$\Theta(n^2)$	$\Theta(n^c)$	$2^{\Theta(n^c)}$
1000	1	≈ 10	1000	≈ 10000	1000000	1000^c	$2^{1000} \approx 10^{301}$
Tempo	1 <i>ns</i>	10 <i>ns</i>	1 μs	10 μs	1 <i>ms</i>	10^{3c-9} s	10^{281} milênios

- $\log_2 \Leftrightarrow \lg$

Modelo de Computação

- Especifica o que a máquina faz em $O(1)$
- word-RAM: Pode acessar qualquer posição da memória em $O(1)$
 - ▶ Não endereça bit a bit, mas byte a byte
 - ▶ Uma palavra é a quantidade mínima entre memória e CPU: 32-bit ou 64-bit
 - ▶ $2^{32} \Rightarrow 4\text{ GB}$
 - ▶ $2^{64} \Rightarrow 20\text{ EB}$ (Exabytes)
- Operações em $O(1)$: aritmética inteira, lógicas, bit a bit, escrever/ler memória
- CPU: opera em tempo $O(1)$ sobre uma quantidade de dados $O(1)$
 - ▶ Operar em quantidade linear toma tempo linear na CPU
 - ▶ Estruturas de Dados: Manter uma grande quantidade de dados (não constante), que suporta um conjunto de operações

Como resolver problemas com algoritmos

- ➊ Reduzir o problema para um que você já sabe como resolver (usando estruturas de dados ou algoritmos)
 - ▶ Busca: array estático/dinâmico, lista encadeada, array ordenado, tabela de espalhamento (hash table), árvore binária balanceada, heap binária
 - ▶ Ordenação: inserção, seleção, bolha, mergesort, heapsort, quicksort, digital (radix), contagem, balde, AVL sort
 - ▶ Caminho mínimo: Busca em largura/profundidade, ordenação topológica, Bellman-Ford, Dijkstra, Johnson, Floyd-Warshall
- ➋ Projetar o seu próprio algoritmo (recursivo)
 - ▶ Força bruta
 - ▶ Divisão e conquista
 - ▶ Programação Dinâmica
 - ▶ Guloso / Incremental

Introdução: Algoritmos e Programação

Yuri Kaszubowski Lopes

UDESC